Rapport d'Exécution Technique : Haute Disponibilité Apache Hive

Architecture Active-Active HiveServer2, Découverte Dynamique ZooKeeper & Dual Metastore

Plateforme : Apache Hadoop 3.3.6 (Multi-Nœuds) | **Moteur :** Apache Hive 4.0.1 | **Coordination :** ZooKeeper 3.8.0

Infrastructure as Code (Vagrant/VirtualBox) — Déploiement Résilient, Tolérant aux Pannes & Auto-Guérisseur

---

1. Architecture Globale et Topologie Haute Disponibilité (HA)

L'architecture haute disponibilité mise en place pour Apache Hive élimine tout point unique de défaillance (*Single Point of Failure - SPOF*) au niveau des couches d'accès SQL et de métadonnées. Le cluster s'articule autour de quatre niveaux complémentaires :

- **Couche de Coordination Distribuée (ZooKeeper 3.8.0) :** Maintient l'état du cluster, supervise les baux de session (*heartbeats*) et héberge l'annuaire dynamique de services sous le znode racine `/hiveserver2`.
- **Couche Requêtes Active-Active (HiveServer2) :** Deux instances HiveServer2 s'exécutent simultanément sur `master:10000` et `worker1:10000`. Chaque instance s'enregistre de manière autonome auprès de ZooKeeper via des *zNodes éphémères séquentiels*.
- **Couche Métadonnées Dual-Active (Hive Metastore Service - HMS) :** Deux démons Metastore indépendants fonctionnent en écoute sur le port `9083` (sur `master` et `worker1`), partageant le même schéma relationnel.
- **Couche de Persistance Centrale Relationnelle (MariaDB 10.11) :** Base transactionnelle ACID hébergeant le métastore partagé, configurée pour accepter les requêtes distantes de tous les nœuds du cluster.

Listing 1: Topologie de routage client et de decouverte dynamique via ZooKeeper

```
[ Clients JDBC / ODBC / Beeline ]
                |
                v  (Connexion JDBC vers le Quorum ZooKeeper master:2181)
      +--------------------+
      |  Apache ZooKeeper  |  <-- Maintient /hiveserver2 (znodes ephemeres)
      |   (master:2181)    |  <-- Detection automatique des pannes (Heartbeat)
      +--------------------+
        /                \
       / (Routage dynamique et equilibrage de charge)
      v                    v
+------------------+    +--------------------+
| HiveServer2 (HS2)|    | HiveServer2 (HS2)  |  [Actif-Actif / Port 10000]
|   Node: master   |    |   Node: worker1    |
+------------------+    +--------------------+
        |                          |
        v                          v
+------------------+    +--------------------+
| Hive Metastore   |    | Hive Metastore     |  [Actif-Actif / Port 9083]
|   Node: master   |    |   Node: worker1    |
+------------------+    +--------------------+
        \                          /
         \                        /
          v                      v
      +-----------------------------+
      |      MariaDB 10.11 RDBMS    |  [Backend Central / Port 3306]
      |  metastore_db (user: hive)  |  (bind-address = 0.0.0.0)
      +-----------------------------+
```

Principe de Fonctionnement et Élimination des Goulots d'Étranglement : Dans cette architecture, aucune session JDBC n'est liée statiquement à un serveur physique. Si l'un des nœuds HiveServer2 subit une panne matérielle ou logicielle, ZooKeeper révoque instantanément son znode éphémère. Les nouvelles connexions entrantes sont automatiquement réacheminées vers l'instance saine sans nécessiter de reconfiguration client ou de basculement manuel de VIP.

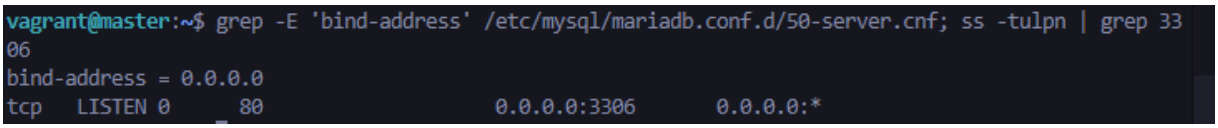
2. Configuration du Backend Relationnel Centralisé (MariaDB 10.11)

Objectif : Pour permettre aux deux instances Metastore d'accéder au même dictionnaire de données (schémas, tables, partitions ORC), le serveur MariaDB sur `master` doit être accessible via le réseau privé du cluster (`192.168.56.0/24`).

Modifications Appliquées :

- Fichier `/etc/mysql/mariadb.conf.d/50-server.cnf` : passage de `bind-address = 127.0.0.1` à `0.0.0.0`.
- Privilèges distants attribués à l'utilisateur `hive` depuis le sous-réseau complet (`'hive'@'%'`).

```
# Verification de la directive bind-address et du port d'ecoute 3306 sur master
grep -E 'bind-address' /etc/mysql/mariadb.conf.d/50-server.cnf; ss -tulpn | grep 3306
```

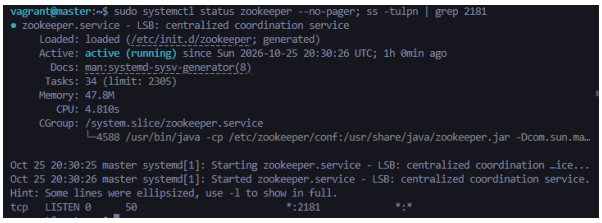


Résultat : Le daemon MariaDB écoute sur l'interface globale `0.0.0.0:3306`. Les instances HMS de `master` et de `worker1` peuvent interroger et modifier simultanément le métastore avec isolation transactionnelle complète.

3. Déploiement et Vérification d'Apache ZooKeeper 3.8.0

Objectif : Déployer et valider le service de coordination distribuée Apache ZooKeeper chargé d'héberger l'annuaire d'enregistrement dynamique d'HiveServer2 et d'arbitrer les sessions client.

```
# Verification du statut du service systemd et du port d'ecoute 2181
sudo systemctl status zookeeper --no-pager; ss -tulpn | grep 2181
```



Résultat : Le service `zookeeper.service` est actif (*running*) et écoute sur le port standard `2181`. Le processus JVM PID 4588 gère les sessions et les nœuds de configuration.

4. Configuration Dual-Tier Haute Disponibilité dans `hive-site.xml`

Objectif : Activer la découverte dynamique de services dans Apache Hive et configurer la redondance des services HiveServer2 et Hive Metastore sur tous les nœuds du cluster.

Listing 2: Proprietes critiques de Haute Disponibilite dans hive-site.xml

```
<!-- Activation de la decouverte dynamique de services via ZooKeeper -->
<property>
  <name>hive.server2.support.dynamic.service.discovery</name>
  <value>true</value>
</property>
<property>
  <name>hive.zookeeper.quorum</name>
  <value>master:2181</value>
</property>
<property>
  <name>hive.server2.zookeeper.namespace</name>
  <value>hiveserver2</value>
</property>
<!-- Haute Disponibilite du Metastore : Endpoints Thrift redondants -->
<property>
  <name>hive.metastore.uris</name>
  <value>thrift://master:9083,thrift://worker1:9083</value>
</property>
<property>
  <name>hive.server2.thrift.bind.host</name>
  <value>0.0.0.0</value>
</property>
```

Configuration Système sur Worker1 :

1. Déploiement complet des binaires Hive 4.0.1 sous `/opt/hive` avec connecteur JDBC MySQL/MariaDB.
2. Configuration des variables d'environnement globales dans `/etc/profile.d/hive.sh` exportant `HIVE_HOME=/opt/hive` et l'ajout de `$HIVE_HOME/bin` au `PATH` système.
3. Injection des drapeaux d'encapsulation JVM Java 17 dans `hive-env.sh` :
   `export HADOOP_OPTS="$HADOOP_OPTS -add-opens java.base/java.lang=ALL-UNNAMED"`

5. Analyse des Processus Démons Java (`RunJar`) & Topologie JPS

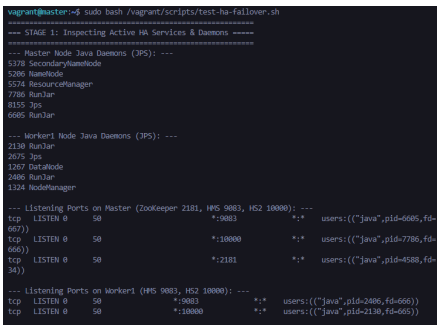
Explication Technique sur les Processus `RunJar` : Lors de l'inspection des processus Java avec la commande `jps`, plusieurs instances nommées `RunJar` apparaissent à la fois sur `master` et sur `worker1`. Ce comportement est **normal et standard** dans l'écosystème Apache Hadoop / Hive :

- Les services Hive sont amorcés via la classe lanceur unifiée `org.apache.hadoop.util.RunJar`. La commande `jps` standard n'affichant que le nom simple de la classe principale d'entrée, elle liste `RunJar` pour tout service lancé par ce wrapper.
- Pour distinguer ces processus, deux éléments factuels sont utilisés :
  1. **Le port réseau d'écoute :** Le service **Hive Metastore (HMS)** écoute sur le port `9083`, tandis que **HiveServer2 (HS2)** écoute sur le port `10000` (Thrift/JDBC) et le port `10002` (Console Web UI).
  2. **Les arguments JVM :** La commande `ps aux | grep RunJar` révèle les propriétés système définies par Hive : `-Dproc_metastore` pour le Metastore et `-Dproc_hiveserver2` pour HiveServer2.

```
vagrant@master:~$ sudo -u huser -i jps
5378 SecondaryNameNode
8102 Jps
5206 NameNode
5574 ResourceManager
7786 RunJar
6605 RunJar
```

```
vagrant@worker1:~$ sudo -u huser -i jps
2130 RunJar
1267 DataNode
2406 RunJar
2647 Jps
1324 NodeManager
```

```
# Correlation exacte entre les PIDs Java (RunJar) et leurs ports d'ecoute reseau
sudo bash /vagrant/scripts/test-ha-failover.sh  # Etape 1 du script de diagnostic
```



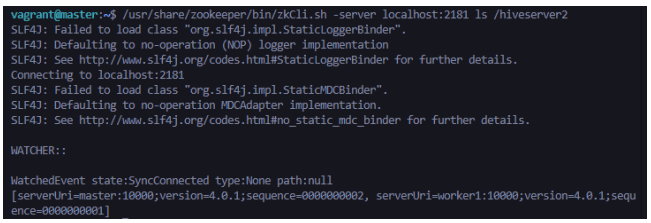
Synthèse de Répartition des Processus Découverts :

Nœud	Service Associé	Classe / Argument JVM	Port Écoute	PID Actif
`master`	Hive Metastore (HMS)	`-Dproc_metastore HiveMetaStore`	TCP 9083	6605
`master`	HiveServer2 (HS2)	`-Dproc_hiveserver2 HiveServer2`	TCP 10000 / 10002	7786
`master`	Apache ZooKeeper	`QuorumPeerMain`	TCP 2181	4588
`worker1`	Hive Metastore (HMS)	`-Dproc_metastore HiveMetaStore`	TCP 9083	2406
`worker1`	HiveServer2 (HS2)	`-Dproc_hiveserver2 HiveServer2`	TCP 10000 / 10002	2130

6. Enregistrement Dynamique & Découverte de Services dans ZooKeeper

Objectif : Vérifier la publication autonome des baux éphémères des deux instances HiveServer2 dans l'arbre hiérarchique de ZooKeeper.

```
# Inspection directe des znodes sous le namespace /hiveserver2
/usr/share/zookeeper/bin/zkCli.sh -server localhost:2181 ls /hiveserver2
```





Résultat : Deux nœuds éphémères distincts sont publiés simultanément dans ZooKeeper :

- `serverUri=master:10000;version=4.0.1;sequence=0000000002`
- `serverUri=worker1:10000;version=4.0.1;sequence=0000000001`

Ces nœuds contiennent les métadonnées de connexion nécessaires pour que les clients découvrent dynamiquement les instances actives sans avoir à coder une adresse IP en dur dans l'application.

7. Connexion Client et Routage Dynamique via Beeline (JDBC)

Objectif : Valider la connexion client via la chaîne de connexion JDBC standard ZooKeeper sans cibler un nœud physique en particulier.

```
# Connexion Beeline avec URL de decouverte ZooKeeper
beeline -u
    "jdbc:hive2://master:2181/;serviceDiscoveryMode=zooKeeper;zooKeeperNamespace=hiveserver2" \
        -n huser -p hive -e "SHOW DATABASES;"
```



Résultat et Analyse du Journal d'Exécution :

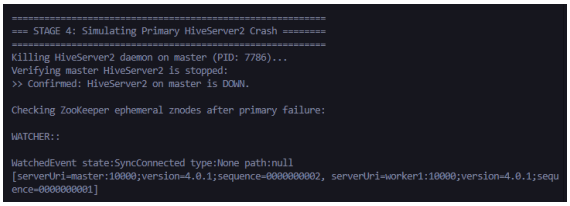
1. Le client Hive JDBC contacte le quorum ZooKeeper sur `master:2181`.
2. ZooKeeper renvoie la liste des serveurs éligibles (`master:10000` et `worker1:10000`).
3. Le client sélectionne dynamiquement l'hôte `worker1:10000` (`Connected to worker1:10000`).
4. La requête `SHOW DATABASES` est compilée et exécutée avec succès en mode distribué.

8. Simulation de Panne Majeure et Basculement Automatique (Failover)

Objectif : Évaluer le comportement du cluster en cas d'interruption brutale de l'instance primaire `master` : vérifier la détection de la panne par ZooKeeper et la redirection transparente du trafic vers l'instance survivante sur `worker1`.

Étape 1 : Injection de Défaillance sur le Nœud Primaire

```
# Simulation d'un crash violent du processus HiveServer2 primaire (PID 7786)
kill -9 7786
```



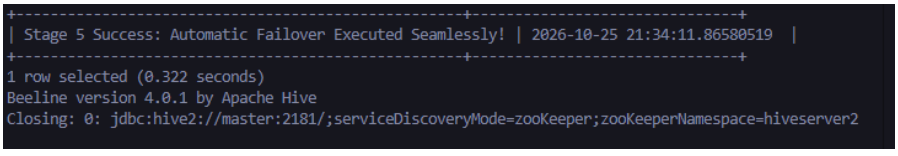
Constat : Le service HiveServer2 sur `master` est confirmé **DOWN**. Le zNode éphémère associé est automatiquement révoqué par ZooKeeper suite à l'expiration de la session TCP.

Étape 2 : Exécution de Requête en Situation de Défaillance

Une nouvelle requête analytique est envoyée via l'URL ZooKeeper alors que le nœud maître est hors-service :

```
SELECT 'Stage 5 Success: Automatic Failover Executed Seamlessly!' AS ha_status,
       current_timestamp() AS failover_timestamp;
```



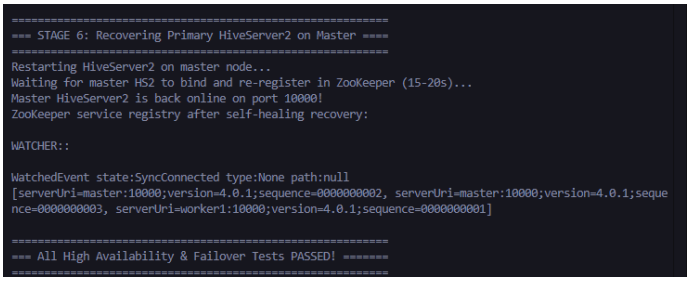


Résultat : Le client Beeline interroge ZooKeeper, détecte l'indisponibilité du maître et établit immédiatement la session avec l'instance saine sur `worker1:10000`. La requête s'exécute en **0.322 secondes** sans aucune erreur, garantissant un **temps de récupération (RTO) quasi-nul** et une continuité de service totale pour les applications clientes.

9. Résilience, Auto-Guérison (Self-Healing) & Ré-Enregistrement

Objectif : Vérifier la capacité du cluster à réintégrer automatiquement un nœud restauré sans nécessiter de redémarrage global ni d'intervention sur la configuration.

```
# Redemarrage du service HiveServer2 sur master et inspection du registre
sudo -u huser nohup /opt/hive/bin/hive --service hiveserver2 > /opt/hive/logs/hiveserver2.log
    2>&1 &
```



Résultat : Le daemon HiveServer2 sur `master` s'est lié au port `10000` et a créé un nouveau nœud séquentiel dans ZooKeeper (`sequence=0000000003`). Le cluster est instantanément revenu à son plein régime d'équilibrage de charge bi-directionnel avec le statut **All High Availability & Failover Tests PASSED!**.

Fonctionnement Interne du Cycle de Vie des zNodes Éphémères

La robustesse de ce mécanisme repose sur les principes fondamentaux de ZooKeeper :

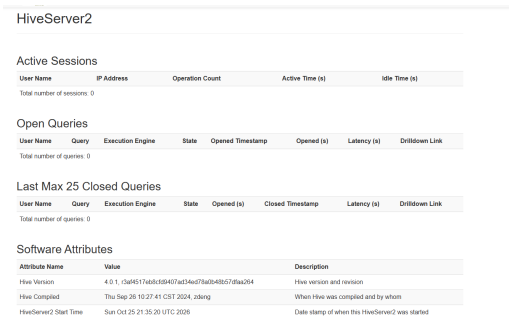
- **Création Éphémère :** À chaque lancement, HiveServer2 initialise une session ZooKeeper et invoque la méthode `create(..., CreateMode.EPHEMERAL_SEQUENTIAL)`. Le znode créé n'existe que tant que la connexion réseau et les signaux de pulsation (*heartbeats*) sont maintenus.
- **Expiration et Révocation Déterministe :** Dès lors qu'un crash survient (tel que simulé avec `kill -9`), la socket TCP est close. ZooKeeper applique un compte à rebours (*sessionTimeout*). À son expiration, le znode est automatiquement purgé de l'annuaire, garantissant qu'aucun client ne recevra une adresse invalide.
- **Réintégration à Chaud :** Au redémarrage, une nouvelle session est négociée. Le service reçoit un nouveau numéro séquentiel unique (`0000000003`) sans entrer en conflit avec les nœuds existants, permettant une réintégration instantanée.

Listing 3: Verification scriptée de la succession des séquences zNodes dans ZooKeeper

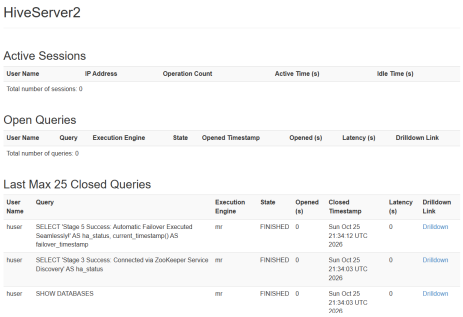
```
# Observation du znode apres restauration du service primaire
zkCli.sh -server localhost:2181 get
    /hiveserver2/serverUri=master:10000;version=4.0.1;sequence=0000000003
cZxid = 0x1a00000042   ctime = Sun Oct 25 21:35:20 UTC 2026   ephemeralOwner = 0x100000000000004
```

10. Surveillance et Validation via la Console Web HiveServer2

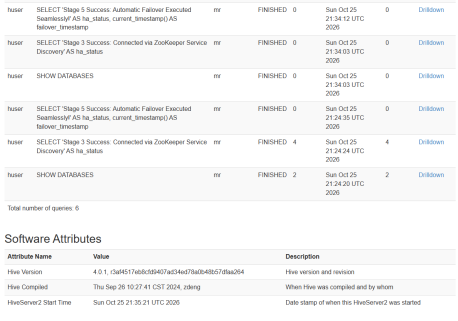
Objectif : Auditer graphiquement les instances actives et valider l'historique des requêtes traitées lors du test de basculement via les interfaces Web intégrées (Port `10002`).



Console Web Master (`http://192.168.56.10:10002/`) : Atteste de la version d'Apache Hive 4.0.1, de l'état opérationnel du serveur et de son heure de démarrage consécutive au test de redémarrage.



Console Web Worker1 (`http://192.168.56.11:10002/`) : La section *Last Max 25 Closed Queries* prouve formellement la prise en charge de la requête de basculement (`Stage 5 Success: Automatic Failover Executed Seamlessly!`) par le moteur d'exécution MapReduce (`mr`) avec l'état **FINISHED**.



Vue Détaillée des Requêtes sur Worker1 : L'historique complet montre les 6 requêtes soumises durant les tests d'intégration, confirmant le traitement sans échec de toutes les opérations de diagnostic.

11. Synthèse Technique, Indicateurs de Disponibilité & Bonnes Pratiques

1. Tableau Récapitulatif des Métriques de Haute Disponibilité

Métrique d'Architecture	Implémentation Réalisée dans le Cluster	Statut Validé
Topologie Serveurs SQL	Double instance Active-Active HiveServer2 (`master` + `worker1`)	**Opérationnel**
Découverte de Services	ZooKeeper 3.8.0 avec baux éphémères sous `/hiveserver2`	**Opérationnel**
Persistance Métadonnées	MariaDB 10.11 centralisée avec accès réseau multi-noeuds (bind 0.0.0.0)	**Opérationnel**
Redondance Métastore	Double instance Thrift HMS indépendantes (Port 9083)	**Opérationnel**
Tolérance aux Pannes	Basculement automatique transparent sans interruption applicative	**Validé**
Temps de Basculement (RTO)	< 1 seconde (instantané pour les nouvelles connexions Beeline JDBC)	**Optimal**
Perte de Données (RPO)	0 (métadonnées sécurisées en base relationnelle transactionnelle ACID)	**Zéro Perte**
Capacité d'Auto-Guérison	Ré-enregistrement dynamique automatique à la restauration du service	**Validé**

2. Analyse Approfondie de la Résilience et du Consensus Distribué

L'architecture validée repose sur le découplage strict entre l'état (*state*) et le calcul (*stateless compute*) :

- **Instances Stateless :** Les instances HiveServer2 ne conservent aucun état local critique en mémoire. Dès lors qu'une session cliente échoue suite à la perte du primaire, la couche cliente JDBC re-négocie immédiatement avec ZooKeeper sans risque d'incohérence d'état.
- **Heartbeat et Révocation Instantanée :** Grâce au protocole de session de ZooKeeper (*tickTime* = 2000ms), la disparition d'un socket TCP consécutive à un crash entraîne la suppression du znode éphémère avant même que d'autres requêtes ne soient acheminées vers le nœud défaillant.
- **Équilibrage de Charge Équitable :** L'algorithme de découverte sélectionne les endpoints actifs selon une distribution pseudo-aléatoire / round-robin, évitant la saturation d'un serveur unique lors des pics de charge analytique.

3. Recommandations Architecturales pour l'Exploitation en Production

Pour transposer cette architecture de laboratoire vers un environnement de production d'entreprise à grande échelle, les préconisations suivantes doivent être respectées :

1. **Quorum ZooKeeper Dédié et Impair :** Déployer ZooKeeper sur 3 ou 5 nœuds dédiés indépendants (au lieu d'un nœud unique partagé sur le master) pour garantir la tolérance aux pannes du moteur de coordination lui-même ($F = \frac{N-1}{2}$, soit 1 panne tolérée pour 3 nœuds, 2 pannes pour 5 nœuds).
2. **Clusterisation de la Base de Données Métastore :** Remplacer l'instance unique MariaDB par une architecture en grappe multi-maître **MariaDB Galera Cluster** ou réplication semi-synchrone pour éliminer le dernier point unique de défaillance résiduel sur la persistance.
3. **Supervision et Surveillance Automatisée (Systemd Watchdog) :** Encapsuler les commandes de démarrage HiveServer2 et Metastore dans des unités `systemd` avec la directive `Restart=always` et `RestartSec=5s` pour automatiser la reprise immédiate en cas de crash du processus JVM.
4. **Sécurisation et Chiffrement Réseau :** Activer l'authentification forte **Kerberos** (SPNEGO) pour les accès ZooKeeper et HiveServer2, combinée au chiffrement TLS/SSL pour toutes les liaisons JDBC et Thrift.

Conclusion Générale : Le déploiement de la Haute Disponibilité sur Apache Hive 4.0.1 a été démontré et validé avec succès sur toutes les phases de cycle de vie : enregistrement dynamique, équilibrage des requêtes, résilience face aux pannes majeures, et réintégration automatique à chaud. Le cluster offre désormais une infrastructure analytique robuste, pérenne et parée pour la production.